

Bitlocker-2

⚠ **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

Jacky has learned about the importance of strong passwords and made sure to encrypt the BitLocker drive with a very long and complex password. We managed to capture the RAM while this drive was opened however. See if you can break through the encryption!

Hints

- Try using a volatility plugin
-

Tools Used

- `file` – Determine file type
 - `strings` – Extract printable character sequences
 - `exiftool` – Read metadata
 - `fdisk` – View partition tables
 - `7z` – Extract compressed files
 - `volatility` – Memory forensics framework (optional)
-

Complete Solution

Step 1: Download and Prepare

Download the disk image `bitlocker-2.dd` and the RAM dump `memdump.mem.gz` from the challenge page.

Step 2: Basic File Inspection

First, verify the file type and general information of the disk image:

```
file bitlocker-2.dd
```

Expected output:

```
bitlocker-2.dd: DOS/MBR boot sector, code offset 0x58+2, OEM-ID "-FVE-FS-",
sectors/cluster 8, reserved sectors 0, Media descriptor 0xf8, sectors/track 63, heads
255, hidden sectors 41531392, FAT (32 bit), sectors/FAT 8160, serial number 0,
unlabeled; NTFS, sectors/track 63, physical drive 0x1fe0, $MFT start cluster 393217,
serial number 02020454d414e204f, checksum 0x41462020
```

The presence of `-FVE-FS-` (Full Volume Encryption File System) indicates this is a **BitLocker-encrypted** drive.

Step 3: Search for Visible Text in the Disk Image

The flag may be present as plain text. Let's try extracting all character strings:

```
strings bitlocker-2.dd | grep -i "picoCTF"
```

Output: (no results)

⚠ No flag appears yet. The data is encrypted, so we cannot read it directly.

Step 4: Inspect Metadata

Use `exiftool` to examine the file metadata for any hidden clues:

```
exiftool bitlocker-2.dd
```

Complete output:

```
ExifTool Version Number      : 13.50
File Name                    : bitlocker-2.dd
```

```
Directory          : .
File Size          : 134 MB
File Modification Date/Time : 2026:05:10 23:51:40+02:00
File Access Date/Time   : 2026:05:13 16:23:36+02:00
File Inode Change Date/Time : 2026:05:10 23:51:59+02:00
File Permissions      : -rw-rw-rw-
Error               : Unknown file type
```

The metadata itself is not suspicious.

Step 5: Examine the Partition Table

Let’s check the disk partition structure:

```
fdisk -l bitlocker-2.dd
```

Output:

```
Disk bitlocker-2.dd: 128 MiB, 134217728 bytes, 262144 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x78787878
```

Device	Boot	Start	End	Sectors	Size	Id	Type
bitlocker-2.dd1		2021161080	4042322159	2021161080	963.8G	78	unknown
bitlocker-2.dd2		2021161080	4042322159	2021161080	963.8G	78	unknown
bitlocker-2.dd3		4294932600	8589899894	4294967295	2T	78	unknown
bitlocker-2.dd4		4294967295	5035196669	740229375	353G	ff	BBT

Note: The partition type `78` is commonly used for **BitLocker encrypted partitions**.

Step 6: Extract the RAM Dump

For BitLocker-2, the password is not needed because the solution relies on extracting the decryption key (FVEK) from the **memory dump** (RAM), bypassing the user password authentication.

First, extract the compressed RAM dump:

```
7z x memdump.mem.gz
```

This creates the file `memdump.mem`.

Step 7: Inspect the RAM Dump

Check the file type:

```
file memdump.mem
```

Output:

```
memdump.mem: Windows Event Trace Log
```

Examine the metadata:

```
exiftool memdump.mem
```

Output:

```
ExifTool Version Number      : 13.50
File Name                    : memdump.mem
Directory                   : .
File Size                    : 2.1 GB
File Modification Date/Time   : 2025:03:10 06:32:30+02:00
File Access Date/Time        : 2026:05:13 16:42:12+02:00
File Inode Change Date/Time   : 2026:05:13 16:42:04+02:00
File Permissions              : -rw-r--r--
Error                        : First 8.2 kB of file is binary zeros
```

Step 8: Search for the Flag in the RAM Dump

Since the RAM was captured while the BitLocker drive was open, the decrypted data (including the flag) may be present in memory as plain text.

Search for the flag pattern:

```
strings memdump.mem | grep -iE "picoCTF"
```

Output:

```
picoCTF{<REDACTED>}  
picoCTF{<REDACTED>}  
picoCTF{<REDACTED>}
```

 **The flag appears directly in the memory dump!**

Final Result

```
picoCTF{<REDACTED>}
```

Note: The actual flag is redacted here. In the real challenge, the `strings` command on the RAM dump will reveal the complete flag.

Key Concepts

- **BitLocker:** A full-disk encryption feature built into Windows.
 - **FVE (Full Volume Encryption):** BitLocker's encryption system, identified by the `-FVE-FS-` OEM-ID.
 - **RAM Dump (Memory Capture):** A snapshot of physical memory, which may contain sensitive data like decryption keys or plaintext files.
 - **FVEK (Full Volume Encryption Key):** The master key used to decrypt the BitLocker volume. It resides in memory while the volume is unlocked.
 - **Memory Forensics:** The analysis of RAM dumps to extract artifacts (processes, network connections, keys, plaintext).
 - **Volatility:** A popular memory forensics framework (mentioned in the hint).
 - **Live Memory Analysis:** Even with strong disk encryption, data is decrypted in RAM while the system is running.
-

Conclusion

The disk image `bitlocker-2.dd` is protected by a strong BitLocker password, making traditional brute-forcing impractical. However, the RAM dump `memdump.mem` was captured while the drive was unlocked. By simply running `strings` on the memory dump and filtering for `picoCTF`, we found the flag in plain text.

This challenge demonstrates a fundamental weakness of disk encryption: **the data must be decrypted in memory while the system is running**. If an attacker can capture RAM (e.g., via cold boot attack, DMA attack, or forensic acquisition), they can bypass the encryption entirely.

💡 **Alternative approach:** Use a Volatility plugin like `bitlocker` or `truecrypt` to extract the FVEK from the memory dump, then use `dislocker` to decrypt the drive. However, the flag was directly visible with `strings`, making the process even simpler.